



Guía de Implementación, Uso y Extensión

ModuTEC

Sistema de Modulación y Demodulación de Señales
en Sistemas Embebidos

Índice general

1. Introducción a ModuTEC	1
1.1. Objetivo de la plataforma	1
1.2. Aplicación académica	1
2. Replicación del Entorno	2
2.1. Componentes requeridos	2
2.1.1. Hardware	2
2.1.2. Software	2
2.2. Esquemático del sistema	3
2.3. Instalación del firmware	4
2.3.1. Etapa 1: Flashear MicroPython	4
2.3.2. Etapa 2: Instalar el firmware de ModuTEC	4
3. Uso de la Herramienta	6
3.1. Detección de la Raspberry Pi Pico	6
3.2. Configuración del sistema	6
3.3. Inicio del monitoreo	8
3.4. Selección de técnicas y ajuste de parámetros	8
4. Extensión de la Plataforma	9
4.1. Creación de nuevas técnicas	9
4.2. Elementos obligatorios del template	9
A. Plantilla de código para nuevas técnicas	11

1. Introducción a ModuTEC

1.1. Objetivo de la plataforma

ModuTEC es una plataforma educativa de código abierto diseñada para la visualización interactiva de técnicas de modulación y demodulación de señales en sistemas embebidos de bajo costo. Su propósito central es proporcionar a estudiantes e investigadores una herramienta concreta y extensible que permita observar en tiempo real el comportamiento de señales analógicas y digitales moduladas, sin requerir equipamiento de laboratorio especializado.

El sistema se compone de dos partes complementarias: un **modulador** que se ejecuta sobre una Raspberry Pi Pico, responsable de generar y procesar las señales y un aplicación **de monitoreo de escritorio**, que recibe los datos vía USB y los visualiza en forma de osciloscopio en tres canales simultáneos: señal mensaje, señal modulada y señal demodulada.

1.2. Aplicación académica

ModuTEC está orientado a cursos universitarios de *Señales y Sistemas* y *Electrónica de Comunicaciones*, donde resulta fundamental que el estudiante comprenda la relación entre parámetros de modulación (frecuencia de la portadora, índice de modulación, tasa de bit, etc.) y como el modificar estos parámetros afecta la forma de la señal resultante.

La plataforma facilita que el estudiante transite desde la teoría matemática de cada técnica hasta su implementación computacional, cerrando el ciclo de aprendizaje con una observación visual directa del resultado.

2. Replicación del Entorno

2.1. Componentes requeridos

2.1.1. Hardware

Para reproducir el entorno de ModuTEC se requieren los siguientes componentes:

- **Raspberry Pi Pico:** Microcontrolador RP2040 o superior con soporte para MicroPython. Se recomienda la versión estándar (sin Wi-Fi), aunque la versión W también es compatible.
- **Potenciómetros:** Se recomiendan 3 unidades de 10 k Ω lineales. Cada potenciómetro mapea a un parámetro de la técnica activa.
- **Interruptor:** Switch de 2 o más posiciones para ciclar entre técnicas disponibles sin intervención desde el monitor.
- **Cable USB-A a USB-C:** Conector para alimentar la Pico, programarla y mantener la comunicación serial con el monitor.
- **Computadora:** Computador externo con sistema operativo Windows, macOS o Linux, con entradas de USB 3.0 o superior.

2.1.2. Software

El computador externo donde se ejecutará la aplicación de monitoreo debe tener instalado el siguiente software:

- **Python 3.11:** Intérprete de Python para ejecutar la aplicación de monitoreo. Versiones 3.12 o superiores también son compatibles. <https://www.python.org/downloads/>.

- **MicroPython:** Librería de firmware para la Raspberry Pi Pico. Se debe instalar la versión estable más reciente disponible en https://micropython.org/download/RPI_PICO/.
- **PyQt5 y PyQtGraph:** Bibliotecas para la interfaz gráfica del monitor y la graficación de alto rendimiento, utilizadas para la visualización en tiempo real de las señales. Su instalación puede realizarse mediante los siguientes comandos:

```
pip install PyQt5 pyqtgraph
```

- **mpremote:** Herramienta de línea de comandos utilizada para transferir archivos a la Raspberry Pi Pico, ejecutar scripts y administrar el dispositivo de forma remota desde la computadora. Su instalación puede realizarse mediante el siguiente comando:

```
pip install mpremote
```

- **Bibliotecas Adicionales:** Recursos necesarios para realizar cálculos matemáticos tanto en el microcontrolador como en la aplicación de monitoreo así como comunicaciones serie. Su instalación puede realizarse mediante el siguiente comando:

```
pip install numpy pyserial
```

2.2. Esquemático del sistema

El esquema eléctrico de ModuTEC es deliberadamente simple. La Raspberry Pi Pico dispone de tres entradas analógicas (GP26, GP27, GP28 también denominadas ADC0, ADC1, ADC2) que se conectan cada una al cursor de un potenciómetro. Los extremos de cada potenciómetro se conectan a 3V3(OUT) y GND respectivamente, de modo que se entrega un voltaje entre 0 y 3.3 V proporcional a la posición.

Adicionalmente, se incorpora un interruptor conectado al pin digital GP5, configurado mediante una resistencia *pull-up* interna. Un terminal del interruptor se conecta a GP5 y el otro a GND. Esta señal se utiliza para seleccionar la técnica de modulación activa dentro del sistema. Por último, el cable USB se conecta del microcontrolador al computador y cumple la función de alimentar el embebido y proporciona el canal de comunicación serial (USB CDC) hacia el monitor de escritorio.

A continuación se presenta un diagrama esquemático del sistema:

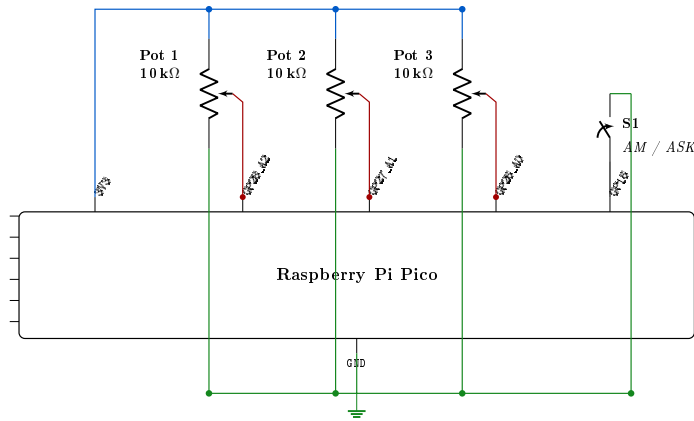


Figura 2.1: Esquemático general del módulo físico de ModuTEC.

2.3. Instalación del firmware

La instalación del firmware en la Raspberry Pi Pico se realiza en dos etapas: primero se flashea MicroPython en la Pico, y luego se transfieren los archivos del proyecto usando la funcionalidad integrada en la aplicación de monitoreo.

2.3.1. Etapa 1: Flashear MicroPython

1. Debemos tener descargado el archivo `.uf2` de MicroPython para Raspberry Pi Pico que se indicó en la sección [2.1.2](#).
2. Mantener presionado el botón `BOOTSEL` de la Pico mientras se conecta el cable USB a la computadora.
3. La Pico aparecerá como una unidad de almacenamiento USB llamada `RPI-RP2`.
4. Copiar el archivo `.uf2` descargado a esa unidad. La Pico se reiniciará automáticamente y quedará corriendo MicroPython.

2.3.2. Etapa 2: Instalar el firmware de ModuTEC

Una vez la Raspberry Pi Pico tiene MicroPython, la transferencia del firmware puede realizarse directamente desde el monitor ModuTEC (`ModuTEC.exe`) mediante la opción *Flash Firmware*. Esta funcionalidad automatiza el proceso de despliegue y copia todos los archivos requeridos hacia el dispositivo, eliminando la necesidad de ejecutar comandos manuales o utilizar herramientas externas.

Durante este proceso, el monitor detecta automáticamente la Raspberry Pi Pico conectada, transfiere los archivos correspondientes al firmware y reinicia el dispositivo para iniciar la ejecución del sistema.

La Figura 2.2 muestra el proceso de despliegue automático utilizando el monitor:

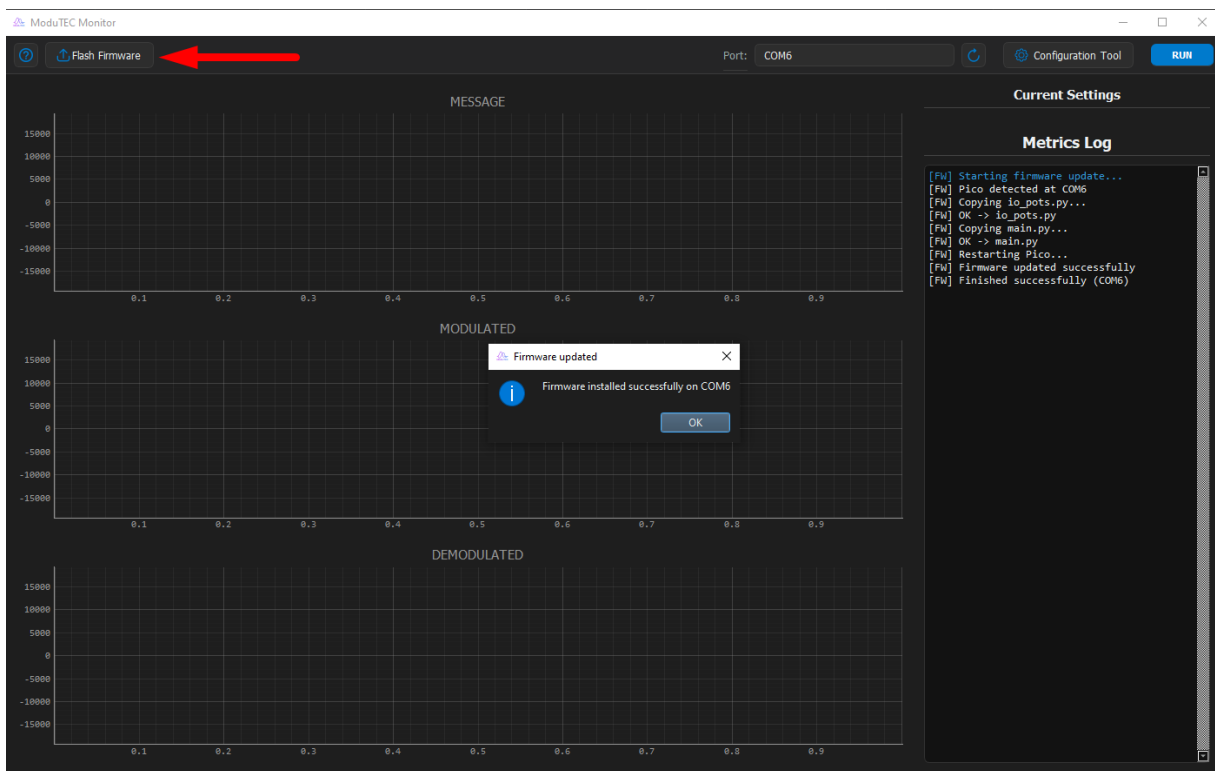


Figura 2.2: Proceso de despliegue automático del firmware utilizando ModuTEC.

3. Uso de la Herramienta

3.1. Detección de la Raspberry Pi Pico

Antes de iniciar el monitoreo, la Raspberry Pi Pico debe encontrarse conectada mediante el cable USB y con el firmware previamente cargado. Una vez abierta la aplicación ModuTEC, se recomienda utilizar la opción *Refresh Ports* para actualizar la lista de puertos seriales disponibles y detectar automáticamente el dispositivo conectado.

Al finalizar la búsqueda, el puerto asociado a la Raspberry Pi Pico aparecerá disponible dentro de la aplicación y podrá ser utilizado para establecer la comunicación con el sistema.

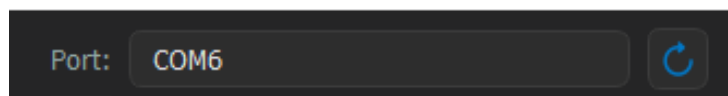


Figura 3.1: Detección de puertos disponibles mediante la opción Refresh Ports.

3.2. Configuración del sistema

ModuTEC incorpora una herramienta de configuración denominada *Config Tool*, la cual permite modificar los parámetros globales del sistema y generar automáticamente el archivo `config.json` utilizado por el firmware.

Desde esta interfaz es posible ajustar parámetros relacionados con la frecuencia de muestreo, tamaño de bloque, amplitud de las señales y rangos de operación de cada técnica implementada.

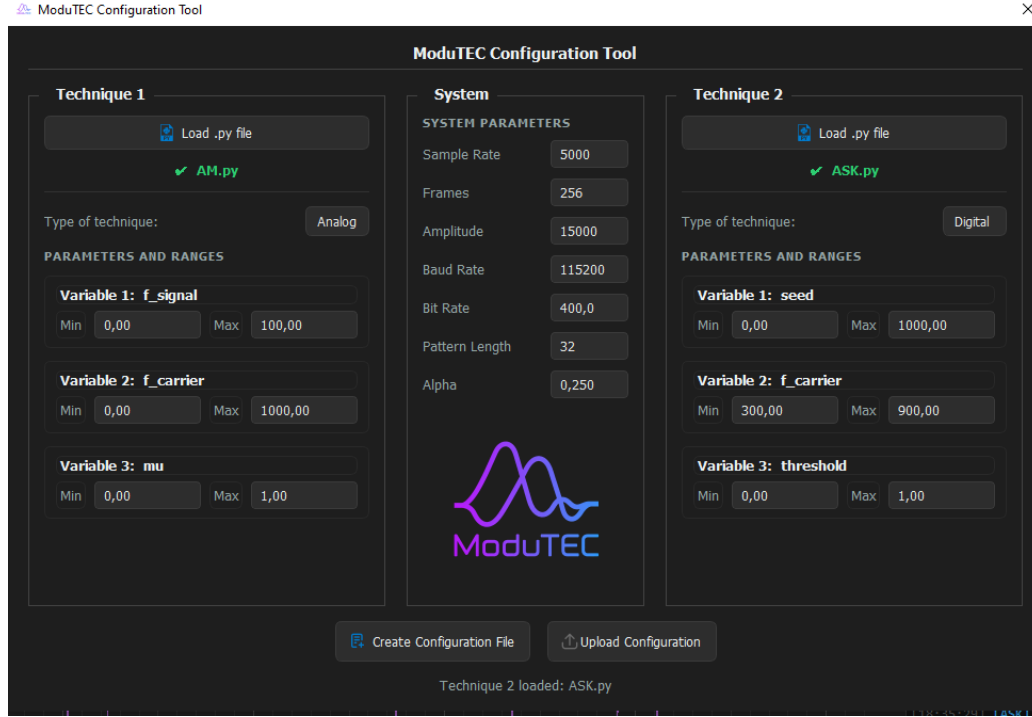


Figura 3.2: Herramienta de configuración de ModuTEC.

Los parámetros globales disponibles se describen en la Tabla 3.1.

Cuadro 3.1: Parámetros globales configurables desde Config Tool.

Campo	Tipo	Descripción
sample_rate	int	Frecuencia de muestreo utilizada para generar y procesar las señales del sistema.
frames	int	Cantidad de muestras procesadas y transmitidas en cada bloque de datos.
amplitude	int	Amplitud máxima utilizada para escalar las señales antes de su transmisión.
baud_rate	int	Velocidad de comunicación serial utilizada entre la Raspberry Pi Pico y el monitor.
bit_rate	float	Tasa de bits empleada por las técnicas de modulación digital.
pattern_length	int	Longitud del patrón pseudoaleatorio utilizado para la generación de datos digitales de prueba.
alpha	float	Coefficiente utilizado por los filtros digitales de demodulación basados en suavizado exponencial.

Adicionalmente, cada técnica posee adicionalmente tres parámetros específicos asociados a los potenciómetros físicos del sistema. El rango de valores para estos parámetros

4. Extensión de la Plataforma

4.1. Creación de nuevas técnicas

Para agregar una nueva técnica de modulación y demodulación a ModuTEC, únicamente se requieren dos pasos:

1. Crear un archivo Python que implemente la técnica siguiendo la estructura estándar proporcionada en el Apéndice [A](#).
2. Registrar la técnica mediante la herramienta *Config Tool*, definiendo sus parámetros y rangos de operación.

Una vez completados estos pasos, la configuración generada puede transferirse al microcontrolador utilizando las herramientas integradas de ModuTEC, quedando la nueva técnica disponible para su ejecución.

4.2. Elementos obligatorios del template

Toda nueva técnica debe ser creada en un archivo de Python con el nombre de la técnica (p. ej. `PSK.py`) y debe incluir los siguientes elementos:

- **PARAM_ORDER**: lista que define el nombre y orden de los parámetros asociados a los tres potenciómetros físicos.
- **State**: estructura utilizada para almacenar variables que deben mantenerse entre bloques de procesamiento, como fases acumuladas, estados de filtros o contadores internos.
- **compute_frame()**: función principal encargada de procesar la señal mensaje, la señal modulada y la señal demodulada para cada bloque de muestras.

Estos elementos constituyen la interfaz mínima requerida por el firmware para cargar y ejecutar dinámicamente cualquier técnica registrada. Una vez respetada esta estructura, la lógica interna de modulación y demodulación puede implementarse libremente según las características de la técnica desarrollada. Por ejemplo, una técnica analógica puede requerir almacenar fases de portadora y estados de filtros, mientras que una técnica digital podría necesitar registros de símbolos, contadores de bits o generadores pseudoaleatorios. Como referencia, el Apéndice [A](#) incluye un archivo base que puede utilizarse como punto de partida para la implementación de nuevas técnicas compatibles con ModuTEC.

A. Plantilla de código para nuevas técnicas

A continuación se presenta el template completo que debe utilizarse como punto de partida para implementar cualquier nueva técnica en ModuTEC. Las secciones marcadas como *# Ejemplo:* deben ser reemplazadas con la lógica específica de la técnica; los comentarios de estructura deben mantenerse para facilitar la revisión y el mantenimiento.

```
1 import math
2 import struct
3
4 # -- Definicion explicita del orden de parametros --
5 PARAM_ORDER = ["param_1", "param_2", "param_3"]
6
7
8 def _clamp(x, lo, hi):
9     return lo if x < lo else hi if x > hi else x
10
11
12 class State:
13     """
14     Estado de TECHNIQUE_NAME.
15     Agregar aqui todas las variables que necesitan persistir
16     entre frames.
17     """
18
19     __slots__ = (
20         # Ejemplo: "phase_c", "lpf_y", etc.
21     )
22
23     def __init__(self):
```

```

24     pass
25     # Ejemplo:
26     # self.phase_c = 0.0
27     # self.lpf_y    = 0.0
28
29
30 def compute_frame(state, params, sys_cfg):
31     """
32     Interfaz estandar ModuTEC.
33
34     Parametros esperados en params (deben coincidir con
35     config.json):
36         - param_1 : descripcion y unidades
37         - param_2 : descripcion y unidades
38         - param_3 : descripcion y unidades
39
40     Retorna:
41         payload_m      : bytearray int16 -- senal mensaje
42         payload_mod     : bytearray int16 -- senal modulada
43         payload_dem     : bytearray int16 -- senal demodulada
44     """
45
46     # -- Extraer parametros del sistema -----
47     FS      = sys_cfg["fs"]
48     N        = sys_cfg["n"]
49     AMP      = sys_cfg["amp"]
50     alpha    = sys_cfg["alpha"]
51     # Tecnicas digitales pueden necesitar ademas:
52     # bit_rate = sys_cfg["bit_rate"]
53     # pattern_len = sys_cfg["pattern_len"]
54
55     # -- Extraer parametros de la tecnica -----
56     param_1 = params["param_1"]
57     param_2 = params["param_2"]
58     param_3 = params["param_3"]
59
60     # -- Inicializar buffers de salida -----
61     payload_m      = bytearray(2 * N)

```

```

62     payload_mod = bytearray(2 * N)
63     payload_dem = bytearray(2 * N)
64
65     # -- Restaurar estado -----
66     # Ejemplo:
67     # phase_c = state.phase_c
68     # lpf_y    = state.lpf_y
69
70     # -- Loop de generacion muestra a muestra -----
71     for i in range(N):
72
73         # Calcular senales (rango normalizado -1..1)
74         msg = 0.0 # senal mensaje
75         s    = 0.0 # senal modulada
76         dem = 0.0 # senal demodulada
77
78         # Escribir muestras en buffers
79         struct.pack_into("<h", payload_m, 2 * i, int(AMP * msg))
80         struct.pack_into("<h", payload_mod, 2 * i, int(AMP * s))
81         struct.pack_into("<h", payload_dem, 2 * i, int(AMP * dem))
82
83         # Actualizar variables de fase u otras internas del loop
84
85     # -- Guardar estado para el proximo frame -----
86     # Ejemplo:
87     # state.phase_c = phase_c
88     # state.lpf_y    = lpf_y
89
90     return payload_m, payload_mod, payload_dem

```

Listing A.1: TECHNIQUE_NAME.py - Template